TOOLKIT · BACKEND ENGINEERING HANDBOOK

# MAVEN COMPLETE HANDBOOK

Build Lifecycle, Dependency Engineering, Plugins, Multi-Stage Docker & Production Workflows

| Phase | Tool ID | Curriculum Scope | Deliverable Type |
|---|---|---|---|
| **I — Developer Foundations** | **Tool 03 of 13 (Maven)** | **Complete Build Engineering** | **Publication Handbook** |
| **Interview Questions** | **Practical Lab** | **Engineering Framework** | **Target Audience** |
| **15 Rigorous Questions** | **Production Build Lab** | **3-Layer Architecture** | **Backend Engineers & SDE-1** |

**Executive Notice**: This handbook is the definitive Java build engineering volume from the *ToolKit* curriculum. It provides comprehensive coverage of Apache Maven's Project Object Model (POM), dependency resolution algorithms, nearest-definition conflict mediation, scopes, lifecycle phases, plugin goal bindings, multi-stage Docker containerization, local vs. CI troubleshooting, 15 core interview questions with deep answers, and production readiness checklists.

# Table of Contents


## LAYER 1 — FUNDAMENTALS & MENTAL MODELS

1. What Exactly Is Maven? (Build Automation & Dependency Management) Section 1
2. Maven versus the JVM, JDK, and Spring Boot (Operational Distinctions) Section 2
3. Understand pom.xml (GAV Coordinates, Packaging, Properties) Section 3
4. Dependencies: Direct vs. Transitive Resolution Graphs Section 4
5. Dependency Versions, Parents, and BOMs (Spring Boot Dependency Management) Section 5
6. Dependency Scopes (compile, provided, runtime, test, system, import) Section 6
7. Maven's Build Lifecycle (Default, Clean, Site Lifecycles & Phase Ordering) Section 7
8. Plugins & Goals: How Maven Performs Work (Compiler, Surefire, Spring Boot) Section 8

## LAYER 2 — PRACTICAL ENGINEERING & WORKFLOWS

9. Install and Verify Maven (JDK & Toolchain Validation) Section 9
10. Essential Maven Commands (Clean, Compile, Test, Package, Verify, Tree) Section 10
11. Standard Java Project Structure (FHS Directory Layout & .gitignore) Section 11
12. Build and Run an Executable Spring Boot JAR Application Section 12
13. Add JUnit 5 Tests (Surefire Plugin, Discovery Conventions, Reports) Section 13
14. Understand Repositories & Local Cache (~/.m2/repository, Mirrors, Central) Section 14
15. Troubleshoot Common Maven Failures (Top 5 Common Errors & Fixes) Section 15
16. Reproducible Builds, Wrapper (mvnw) & Dependency Hygiene Section 16
17. Maven in CI/CD & Multi-Stage Docker Builds (Minimal Runtime Images) Section 17

## LAYER 3 — PRODUCTION ENGINEERING & INTERVIEW MASTERY

18. What Happens Internally When a Maven Build Runs (Lifecycle Execution Sequence) Section 18
19. Dependency Conflicts: Nearest-Definition Rule & Runtime Classpath Errors Section 19
20. Production Failure Scenario: Works Locally but Fails in CI (7-Step Triage) Section 20
21. Maven Interview Questions — 15 Essential Questions with Detailed Answers Section 21
22. Hands-on Engineering Lab — Verification, Dependency Trees & Packaging Section 22
23. Final Mastery Checklist & Completion Protocol Section 23

# Maven — Backend Engineering Tools & Infrastructure Handbook

**Phase:** I — Developer Foundations

**Chunk:** 1 of 1

**Source file:** `content/maven/01-maven-fundamentals-build-workflows-production.md`

---

### Learning Objectives

By the end of this chapter, you should be able to:

- Explain how Maven builds a Java backend application.
- Understand `pom.xml`, dependencies, scopes, plugins and repositories.
- Execute the Maven lifecycle and troubleshoot failed builds.
- Manage dependency versions and produce reproducible builds.
- Connect Maven with Spring Boot, JUnit 5, Docker and CI/CD.
- Answer practical SDE-1 interview questions about build engineering.

---

# Layer 1 — Fundamentals & Mental Models

## 1. What exactly is Maven?

Imagine you're building a Spring Boot backend.

Your application depends on Spring Web, Spring Data JPA, PostgreSQL drivers and JUnit. You need to compile Java files, run tests, package the application and prepare it for deployment.

Without a build tool, you'd have to manage dependencies and orchestrate these steps yourself.

**Apache Maven is a build automation and dependency management tool for Java projects.**

It provides a standardized project structure, a dependency resolution system and a lifecycle for compiling, testing and packaging software.

### The Maven Mental Model

Maven doesn't replace Java or Spring Boot. It orchestrates the build and resolves the libraries your project needs.

---

## 2. Maven versus the JVM, JDK and Spring Boot

| Technology | Primary responsibility | Operational Distinction |
|---|---|---|
| **JDK** | Java compiler, development tools and runtime components | Provides `javac`, `java`, `jcmd`, headers and developer binaries |
| **JVM** | Executes Java bytecode | Runs compiled `.class` files in memory; platform-specific implementation |
| **Maven** | Manages dependencies and orchestrates builds | Declarative build system executing plugins across lifecycle phases |
| **Spring Boot** | Simplifies building and running Spring applications | Framework opinionated defaults, auto-configuration and starters |
| **JUnit 5** | Provides Java testing APIs and test execution support | Testing framework executed by Maven Surefire / Failsafe plugins |

A useful distinction: **Maven builds your application; the JVM runs Java bytecode.**

---

## 3. Understand `pom.xml`

POM stands for **Project Object Model**. It is the XML file Maven uses to describe the project.

A minimal example:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>orders-api</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

</project>
```

Important fields:

- **`groupId`**: identifies the organization or project namespace (e.g. reverse domain **`com.example.billing`**).
- **`artifactId`**: identifies the specific artifact or project module (e.g. **`orders-api`**).
- **`version`**: identifies the project version (e.g. **`1.0.0-SNAPSHOT`** or **`1.0.0.RELEASE`**).
- **`packaging`**: determines the primary package type, commonly **`jar`**, **`war`**, or **`pom`**.
- **`properties`**: centralizes reusable configuration values and build settings.

Together, the coordinates are often represented as:

**`groupId:artifactId:version`** (often called GAV coordinates).

For this example:

**`com.example:orders-api:1.0.0`**

---

## 4. Dependencies: how your application gets libraries

Suppose your application needs Spring Web.

You declare the dependency in **`pom.xml`**:

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

This declaration tells Maven which artifact your project requires. Maven resolves the artifact and its transitive dependencies.

### What are transitive dependencies?

Suppose library A depends on B, and B depends on C.

Your project declares A. Maven may resolve B and C automatically as part of the dependency graph.

```
Your application
      ■
      ▼
   Library A
      ■
      ▼
   Library B
      ■
      ▼
   Library C
```

This is convenient, but it also means a dependency can enter your application without appearing directly in your own dependency declarations.

That's why dependency inspection, version management and vulnerability scanning matter in production engineering.

---

## 5. Dependency versions and Spring Boot

For Spring Boot projects, the Spring Boot parent POM or its dependency-management BOM can provide a compatible set of dependency versions.

Example using a Spring Boot parent:

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
    <relativePath/>
</parent>
```

With this parent, many Spring-managed dependencies can omit explicit versions because the parent manages them.

Alternatively, projects can import the Spring Boot dependency-management BOM without inheriting from the parent:

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-dependencies</artifactId>
            <version>3.2.0</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

**Important distinction:**

- A **dependency declaration** (`<dependencies>`) says what your project uses and pulls into the build.
- A **dependency management block** (`<dependencyManagement>`) controls versions and configurations without actually adding artifacts until declared.
- A **BOM (Bill of Materials)** is a POM that centralizes a curated set of compatible dependency versions.

Avoid independently upgrading several related Spring libraries without understanding their compatibility.

---

## 6. Dependency scopes

Scopes determine where a dependency is available across the compile, test, and runtime classpaths.

| Scope | Classpath Availability | Transitive? | Main Purpose & Example |
|---|---|---|---|
| `compile` | Compile, Test, Runtime | Yes | Default scope; required to build and run (e.g. `spring-boot-starter-web`) |
| `provided` | Compile, Test (Not Runtime) | No | Supplied by the runtime container or JDK (e.g. `servlet-api`, `lombok`) |
| `runtime` | Test, Runtime (Not Compile) | Yes | Required only for execution, not compilation (e.g. PostgreSQL JDBC driver) |
| `test` | Test Only | No | Used for compiling and running test suites (e.g. `junit-jupiter`, `mockito`) |
| `system` | Compile, Test | Yes | Local filesystem path via `<systemPath>`; generally avoided due to non-portability |
| `import` | POM dependencyManagement | N/A | Imports dependency version definitions from an external BOM POM |

Example:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
```

JUnit belongs in the `test` scope because production application code normally shouldn't depend on or package the test framework.

---

## 7. Maven's build lifecycle

A lifecycle is a sequence of build phases. Maven executes earlier phases in that lifecycle when you request a later phase.

### Default Lifecycle — Key Phases

| Phase | Description & Lifecycle Responsibility |
|---|---|
| `validate` | Check that the project configuration is valid and all necessary information is available |
| `compile` | Compile the main production source code (`src/main/java`) into target classes |
| `test` | Compile test sources (`src/test/java`) and execute unit tests using the configured test tooling |
| `package` | Take the compiled bytecode and package it into its distributable format (e.g. JAR, WAR) |
| `verify` | Run additional integration tests and quality checks configured for the verification stage |

| Phase | Description & Lifecycle Responsibility |
|---|---|
| `install` | Install the packaged artifact into the local repository cache (`~/.m2/repository`) |
| `deploy` | Publish the finalized artifact to a configured remote corporate artifact repository |

These are phases in Maven's **default lifecycle**. Additional independent lifecycles exist:

- `clean`: Handles project cleaning (`pre-clean`, `clean`, `post-clean`) to remove the `target/` directory.
- `site`: Generates project documentation and reports (`site`, `site-deploy`).

### Why does `mvn package` run tests?

Because `package` occurs after `test` in the default lifecycle. When you ask Maven to execute a phase, Maven runs all preceding phases in strict order:

`validate` → `compile` → `test` → `package`

Similarly, `mvn verify` includes `clean` only if explicitly requested (`mvn clean verify`), running all phases up to `verify`.

This gives you a standardized, reproducible build pipeline instead of a collection of fragile manual shell scripts.

---

## 8. Plugins: how Maven performs work

Maven's core defines lifecycle phases, but **plugins bind goals to phases and perform most of the actual build work**.

Examples:

- **Compiler Plugin** (`maven-compiler-plugin`): compiles Java source files during the `compile` phase.
- **Surefire Plugin** (`maven-surefire-plugin`): runs unit tests during the `test` phase.
- **Failsafe Plugin** (`maven-failsafe-plugin`): runs integration tests during `integration-test` and verifies during `verify`.
- **JAR Plugin** (`maven-jar-plugin`): packages compiled classes into a standard JAR during `package`.
- **Spring Boot Maven Plugin** (`spring-boot-maven-plugin`): repackages the standard JAR into an executable, fat Spring Boot JAR with embedded Tomcat and launcher classes.

### Mental Model

- **Lifecycle:** *When* build work happens.
- **Plugin:** *Which* tool performs the work.
- **Goal:** The *specific operation* executed by a plugin (e.g. `compiler:compile`, `surefire:test`, `spring-boot:repackage`).

A plugin goal is bound to a lifecycle phase either by default packaging bindings or explicit POM configuration.

---

# Layer 2 — Practical Engineering & Workflows

## 9. Install and verify Maven

Check your Java installation:

```
java -version
javac -version
```

Check Maven:

```
mvn -version
```

If Maven isn't installed, install it using an appropriate package manager or the official Apache Maven distribution. On macOS, Homebrew is one option:

```
brew install maven
```

Verify that Maven can locate the intended JDK. The Java version shown by `mvn -version` is particularly useful when troubleshooting a build that works in your IDE but fails in the terminal.

---

## 10. Maven commands you should know

| Command | Primary Engineering Purpose | Typical Usage Context |
|---|---|---|
| `mvn clean` | Remove generated build output (`target/` directory) | Reset build state before fresh compilation |
| `mvn compile` | Compile production source code | Quick syntax and type verification |
| `mvn test` | Run the configured unit test phase | Fast local test execution loop |
| `mvn package` | Build the distributable artifact (JAR/WAR) | Local packaging before running executable |
| `mvn verify` | Run full verification (compile, test, package, integration tests) | Standard pre-commit verification command |
| `mvn install` | Install artifact into local cache (`~/.m2/repository`) | Sharing local library between projects |
| `mvn dependency:tree` | Inspect full tree of direct and transitive dependencies | Debugging version conflicts and classpath issues |
| `mvn help:effective-pom` | Print effective POM with all inherited parent configurations | Inspecting resolved properties and plugin defaults |
| `mvn -U clean verify` | Force refresh of snapshots/release metadata and verify | Resolving stale remote snapshot caches |

> ***Operational Tip:*** `-U` *is primarily useful when Maven should check for updated snapshot dependencies and missing release metadata from remote repositories. It is not a universal repair command for syntax or logic bugs.*

---

## 11. Standard Java project structure

A conventional Maven project strictly follows the Standard Directory Layout:

```
orders-api/
■■■ pom.xml                              # Project configuration
■■■ src/
■   ■■■ main/
■   ■   ■■■ java/                         # Production Java source code
■   ■   ■   ■■■ com/example/orders/
■   ■   ■       ■■■ OrdersApplication.java
■   ■   ■       ■■■ service/OrderService.java
■   ■   ■■■ resources/                    # Application configuration (application.prope
rties/yml)
■   ■■■ test/
■       ■■■ java/                         # Test Java source code
■       ■   ■■■ com/example/orders/
■       ■       ■■■ OrderServiceTest.java
■       ■■■ resources/                    # Test-specific properties, mock payloads, SQL
 fixtures
■■■ target/                               # Ephemeral generated build output (classes, J
ARs, reports)
```

- **`src/main/java`**: production Java source.
- **`src/main/resources`**: application configuration, static files, and templates.
- **`src/test/java`**: test source code.
- **`src/test/resources`**: test configurations and mocks.
- **`target/`**: compiled `.class` files, generated test reports, and final `.jar`.

**Rule:** Never commit **`target/`** to Git. Add **`target/`** to your **`.gitignore`**. It can always be deterministically reproduced from source code and **`pom.xml`**.

---

## 12. Build a Spring Boot application

A typical workflow:

```
git status
mvn clean verify
```

If verification succeeds, package the application:

```
mvn package
```

Locate the artifact:

```
ls -lh target/*.jar
```

For an executable Spring Boot JAR, you can start it with:

```
java -jar target/orders-api-1.0.0.jar
```

The filename depends on the artifact configuration. If the project doesn't configure an executable Spring Boot JAR with **`spring-boot-maven-plugin`**, the JAR will not contain the embedded server or **`Main-Class`** manifest header.

**Security Rule:** Keep secrets and environment-specific configuration outside the artifact where practical. Never hardcode database passwords, JWT signing keys, or cloud credentials directly into **`pom.xml`** or committed configuration files.

---

## 13. Add JUnit 5 tests

A Spring Boot parent commonly manages compatible versions for test dependencies. A typical dependency is:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

A simple unit test:

```
package com.example.orders;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class PriceCalculatorTest {

    @Test
    void calculatesTotalPrice() {
        int quantity = 3;
        int unitPrice = 100;

        assertEquals(300, quantity * unitPrice);
    }
}
```

Run tests with:

```
mvn test
```

If tests aren't discovered:

1. Verify class name ends in `Test` or `Tests` (default Surefire discovery convention).
2. Check that `@Test` imports `org.junit.jupiter.api.Test` (JUnit 5), not `org.junit.Test` (JUnit 4 legacy).
3. Review Surefire XML reports generated under `target/surefire-reports/`.

---

## 14. Understand repositories and the local cache

Maven retrieves artifacts from repositories:

- **Local repository:** a directory cache on your machine, typically under `~/.m2/repository`.
- **Remote repository:** a server from which Maven downloads dependencies or to which authorized builds publish artifacts.
- **Central repository:** Maven Central (`repo.maven.apache.org`), the default public registry for open-source Java artifacts.
- **Private / Corporate repository:** an internal repository manager (e.g. Sonatype Nexus, JFrog Artifactory) hosting proprietary enterprise artifacts and caching approved external libraries.

Inspect your local cache:

```
ls ~/.m2/repository
```

If dependency resolution fails, investigate coordinates, repository definitions in `~/.m2/settings.xml`, corporate VPN access, HTTP proxies, and specific error messages before blindly deleting `~/.m2/repository`.

A cached artifact accelerates builds, but the local cache should not be mistaken for the authoritative source of your project's dependency declarations.

---

## 15. Troubleshoot common Maven failures

### Failure 1: `mvn: command not found`

- **Root Cause**: Maven is not installed or its `bin` directory is missing from your system `$PATH`.
- **Diagnosis**:

```
which mvn
echo "$PATH"
```

- **Remediation**: Install Maven (`brew install maven` or package manager) or set `export PATH="/opt/apache-maven/bin:$PATH"` in your shell profile.

### Failure 2: Unsupported Java version / invalid target release

- **Root Cause**: The JDK used by Maven does not match `<maven.compiler.release>` or `<java.version>` in `pom.xml`.
- **Diagnosis**:

```
java -version
mvn -version
```

- **Remediation**: Align `JAVA_HOME` with the required version (e.g. `export JAVA_HOME=$(/usr/libexec/java_home -v 21)`).

### Failure 3: Dependency cannot be resolved (`Could not find artifact...`)

- **Root Cause**: Misspelled `groupId`/`artifactId`, incorrect version, private repository credentials missing, or network proxy issue.
- **Diagnosis**:

```
mvn -U dependency:tree
```

- **Remediation**: Verify GAV coordinates on Maven Central; verify `<repositories>` and `<servers>` authentication in `~/.m2/settings.xml`.

### Failure 4: Tests pass in the IDE but fail in Maven

- **Root Cause**: IDE uses different compiler target, cached test artifacts, ambient environment variables, or different timezone/locale.
- **Diagnosis**:

```
mvn clean test
cat target/surefire-reports/*.txt
```

- **Remediation**: Verify test independence from execution order and eliminate dependencies on local hardcoded filesystem paths.

### Failure 5: `NoSuchMethodError` or `ClassNotFoundException` at runtime

- **Root Cause**: Dependency conflict where an incompatible transitive dependency version was selected via the nearest-definition rule.
- **Diagnosis**:

```
mvn dependency:tree -Dverbose
```

- **Remediation**: Trace which dependency pulls the incompatible version; apply an explicit `<exclusion>` or force the correct version in `<dependencyManagement>`.

---

## 16. Reproducible builds and dependency hygiene

A build should produce predictable, identical outcomes across developer workstations and CI runners.

### Best Practices:

1. **Pin plugin versions**: Always specify explicit `<version>` tags for all build plugins.
2. **Use `<dependencyManagement>`**: Centralize versions in a parent POM or BOM.
3. **Commit the Maven Wrapper**: Always commit `mvnw`, `mvnw.cmd`, and `.mvn/wrapper/` into version control.
4. **Consistent JDK**: Ensure CI runners execute the exact same major/minor JDK distribution as local developers.
5. **Avoid version ranges**: Do not use dynamic version ranges like `[1.0, 2.0)` or `RELEASE` in production POMs.
6. **Vulnerability scanning**: Use plugins like OWASP Dependency-Check or Snyk to audit dependencies during CI.

### Using the Maven Wrapper

The Maven Wrapper lets a project lock and bootstrap its exact required Maven version:

```
./mvnw clean verify
```

On Windows:

```
mvnw.cmd clean verify
```

The wrapper eliminates the requirement for engineers or CI workers to have Maven pre-installed globally.

---

## 17. Maven in CI/CD and Docker

A standard production backend delivery pipeline:

```
Git Commit pushed to main
          ■
          ▼
CI Runner checks out code
          ■
          ▼
Maven compiles and runs unit tests (`./mvnw clean test`)
          ■
          ▼
Maven packages application (`./mvnw -B package -DskipTests`)
          ■
          ▼
Docker multi-stage build creates minimal JRE runtime image
          ■
          ▼
Automated container scan & staging deployment
```

### Production Multi-Stage Dockerfile

```
# Stage 1: Build & Package
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

# Cache dependency layer
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copy source and build
COPY src ./src
RUN mvn -B clean package -DskipTests

# Stage 2: Minimal Production JRE Image
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Create unprivileged application user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

COPY --from=build --chown=appuser:appgroup /app/target/*.jar app.jar

USER appuser
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Multi-stage builds prevent compiler tools, Maven caches, and source code from leaking into the lightweight production runtime container image.

---

# Layer 3 — Production Engineering & Interview Mastery

## 18. What happens when a Maven build runs?

Suppose you execute:

```
./mvnw clean verify
```

Step-by-step internal execution:

1. **Bootstrap**: The `mvnw` shell script checks `.mvn/wrapper/maven-wrapper.properties` and downloads the designated Maven distribution if absent.
2. **Effective POM construction**: Maven merges your `pom.xml`, the Super POM (built into Maven), parent POMs, and active profiles into a single unified Project Object Model.
3. **Dependency graph calculation**: Maven traverses declared and transitive dependencies, builds the directed acyclic graph (DAG), and resolves conflicts using the nearest-definition rule.
4. **Lifecycle planning**: Maven binds configured plugin goals to phases for both the `clean` and `default` lifecycles up to `verify`.
5. **Execution**:

- `clean:clean`: deletes `target/`.
- `resources:resources`: copies `src/main/resources` to `target/classes`.
- `compiler:compile`: invokes `javac` on `src/main/java`.
- `resources:testResources`: copies test resources.
- `compiler:testCompile`: compiles `src/test/java`.
- `surefire:test`: executes unit tests and writes reports.
- `jar:jar` / `spring-boot:repackage`: packages classes into JAR archive.
- `failsafe:integration-test` / `failsafe:verify`: executes integration tests.

6. **Exit status**: Maven terminates with exit code `0` on success, or nonzero on failure.

---

## 19. Dependency conflicts: why builds compile but fail at runtime

Suppose your project depends on two third-party libraries:

```
orders-api
   ■
   ■■■ library-A:1.0
   ■      ■
   ■      ■■■ common-lib:1.4  (Depth: 2)
   ■
   ■■■ library-B:2.0
          ■
          ■■■ common-lib:2.0  (Depth: 2)
```

Maven resolves conflicts using the **Nearest-Definition Rule**:

- Whichever version is closest to the root of the dependency tree wins.
- If two conflicting versions are at the exact same depth, the **first declared in the POM** wins.

If `common-lib:1.4` is selected, but `library-B` calls a method introduced only in version `2.0`, the compiler may succeed (if compiled against separate interfaces), but at runtime the JVM throws:

```
java.lang.NoSuchMethodError: com.example.common.Utils.executeV2()V
```

### Remediation Workflow:

1. Run `mvn dependency:tree -Dincludes=com.example:common-lib` to identify conflict branches.
2. Explicitly declare the required version in `<dependencyManagement>` to override transitive resolution:

```
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>com.example</groupId>
                <artifactId>common-lib</artifactId>
                <version>2.0</version>
            </dependency>
        </dependencies>
    </dependencyManagement>
```

---

## 20. Production failure scenario: CI builds locally but fails in CI

**Situation:** Your application builds cleanly on your developer MacBook, but the CI pipeline consistently fails.

### Systematic Triage Sequence:

1. **JDK Distribution Mismatch**: Developer uses JDK 21.0.2; CI uses JDK 17. Verify `<maven.compiler.release>21</maven.compiler.release>` in POM.
2. **Maven Wrapper Missing**: Developer uses global `mvn` (Maven 3.9.6); CI uses an outdated Maven 3.6. Enforce `./mvnw` in CI.
3. **Uncommitted Files**: A new class or resource exists locally on the MacBook but was not added (`git add`) to Git.
4. **Case Sensitivity**: macOS filesystem (APFS) is case-insensitive by default (`User.java` matches `user.java`); Linux CI runner filesystem (ext4) is strictly case-sensitive.
5. **Timezone/Locale Assumptions**: Tests asserting formatted dates fail because local developer timezone is IST/PST while CI runs in UTC.
6. **Hardcoded Paths**: Tests rely on `/Users/name/...` or `C:\...` rather than relative classpath resources (`getClass().getResourceAsStream(...)`).
7. **Flaky Concurrency**: Parallel test execution succeeds locally on 8 cores but deadlocks on a 2-core CI runner.

---

## 21. Maven interview questions — 15 essential questions

### Q1. What is Maven?

**Answer**: Apache Maven is a declarative build automation and dependency management tool for Java applications. It standardizes project layout, coordinates dependency resolution from repositories, and executes structured build lifecycles via plugins.

### Q2. What is `pom.xml`?

**Answer**: The Project Object Model (POM) is an XML file that serves as Maven's fundamental unit of configuration. It contains project coordinates (GAV), dependencies, build plugins, active profiles, and compiler properties.

### Q3. What is the difference between a dependency and a plugin?

**Answer**: A **dependency** is a library or compiled bytecode artifact included on the application's classpath for compilation, testing, or execution. A **plugin** is a build utility (e.g. Compiler, Surefire, Spring Boot plugin) that executes specific tasks during lifecycle phases.

### Q4. What is a Maven lifecycle?

**Answer**: A Maven lifecycle is an ordered sequence of phases that define the build workflow. Executing a phase causes Maven to sequentially run every phase that precedes it within that lifecycle. The three built-in lifecycles are `default`, `clean`, and `site`.

### Q5. Why does `mvn package` run tests?

**Answer**: In Maven's default lifecycle, the `test` phase occurs immediately before the `package` phase (`validate` → `compile` → `test` → `package`). Maven strictly executes all preceding phases in order before executing the requested target phase.

### Q6. What is the difference between `package`, `install` and `deploy`?

**Answer**:

- `package`: compiles code, runs tests, and archives bytecode into its target distribution format (e.g. `.jar` in `target/`).
- `install`: copies the packaged artifact into the local workstation cache (`~/.m2/repository`).
- `deploy`: uploads the final artifact to an external remote repository (e.g. Nexus, Artifactory) for team-wide sharing.

### Q7. What are transitive dependencies?

**Answer**: Dependencies that are required by the libraries your project directly declares. Maven traverses the dependency tree and automatically resolves and downloads these secondary dependencies.

### Q8. What is a dependency scope?

**Answer**: A configuration setting (`compile`, `provided`, `runtime`, `test`, `system`, `import`) that restricts where a dependency is included across Maven's distinct classpaths (compilation, testing, runtime) and whether it is transitively inherited.

### Q9. What is a BOM (Bill of Materials)?

**Answer**: A special POM file packaging (`<packaging>pom</packaging>`) that defines an aligned set of library versions inside a `<dependencyManagement>` section. Other projects import it with `<scope>import</scope>` to standardize dependency versions without inheriting from a parent POM.

### Q10. What is the Maven Wrapper?

**Answer**: A set of lightweight repository-committed scripts (`mvnw`, `mvnw.cmd`, `.mvn/wrapper/`) that download and execute a verified, specific version of Maven, ensuring identical build environments across all developer machines and CI pipelines.

### Q11. What is the local Maven repository?

**Answer**: A directory cache on the developer's machine (default: `~/.m2/repository`) where Maven stores all downloaded remote dependencies, plugins, and locally installed project artifacts.

### Q12. What is the effective POM?

**Answer**: The complete, fully merged POM in memory resulting from combining the project `pom.xml`, parent POM hierarchies, the internal Maven Super POM, and active configuration profiles. Inspectable via `mvn help:effective-pom`.

### Q13. How would you investigate and resolve a dependency conflict?

**Answer**:

1. Run `mvn dependency:tree -Dverbose` to visualize conflicting versions and nearest-definition selections.
2. Inspect runtime exceptions (`NoSuchMethodError`, `ClassNotFoundException`).
3. Explicitly declare the required version in `<dependencyManagement>` or add a targeted `<exclusion>` inside the conflicting `<dependency>`.

### Q14. Why might tests pass locally in an IDE but fail in Maven?

**Answer**: IDEs often compile incrementally, ignore Surefire configuration flags, use ambient developer environment variables, have different default JVM/JDK versions, or fail to enforce strict phase-ordering that Maven requires.

### Q15. How do you ensure builds are reproducible and secure?

**Answer**: Commit the Maven Wrapper, enforce `<maven.compiler.release>`, pin all plugin versions explicitly, manage versions via BOMs, avoid version ranges, run dependency vulnerability scans, and maintain parity between local and CI JDK environments.

---

## 22. Hands-on engineering lab

Complete the following practical tasks in a Java/Spring Boot project.

### Maven Practical Checklist

- [ ] Verify Java, javac and Maven versions (`java -version`, `javac -version`, `mvn -version`).
- [ ] Inspect the POM and explain the project GAV coordinates (`groupId`, `artifactId`, `version`).
- [ ] Run `./mvnw clean test` or `mvn clean test` and verify test execution in console output.
- [ ] Run `./mvnw clean verify` and inspect output in the `target/` directory.
- [ ] Inspect dependencies using `mvn dependency:tree` to trace transitive resolution paths.
- [ ] Generate the effective POM (`mvn help:effective-pom`) and identify inherited compiler plugin settings.
- [ ] Create an intentional failing unit test (`assertEquals(1, 2)`), observe build failure, and fix it.
- [ ] Package the application with `mvn package` and inspect the generated artifact (`ls -lh target/*.jar`).
- [ ] Run the executable Spring Boot JAR with `java -jar target/*.jar` and verify startup logs.
- [ ] Document one likely cause of a local-versus-CI build mismatch and the corresponding preventive action.

---

## 23. Final mastery checklist

You are ready to move on when you can explain, without memorizing definitions:

- [ ] How Maven resolves direct and transitive dependencies.
- [ ] How `pom.xml`, dependency management and plugins interact.
- [ ] Why lifecycle phases run in a particular order.
- [ ] How `test`, `package`, `verify`, `install` and `deploy` differ.
- [ ] How to debug a failed compilation or dependency resolution.

- [ ] How to investigate runtime dependency conflicts.
- [ ] Why the Maven Wrapper and consistent JDK configuration matter.
- [ ] How Maven fits into CI/CD and a Docker build.
- [ ] How to make a build more reliable and secure.

---

## Completion Protocol

1. Authoritative source file: `content/maven/01-maven-fundamentals-build-workflows-production.md`.
2. All three layers present: Fundamentals & Mental Models, Practical Engineering & Workflows, and Production Engineering & Interview Mastery.
3. Single final Maven handbook PDF deliverable: `docs/maven/BEH_MAVEN_COMPLETE_HANDBOOK.pdf`.
4. Update `README.md` progress tracker.
5. Commit using exact message: `Maven done`.